PCA

Introduction

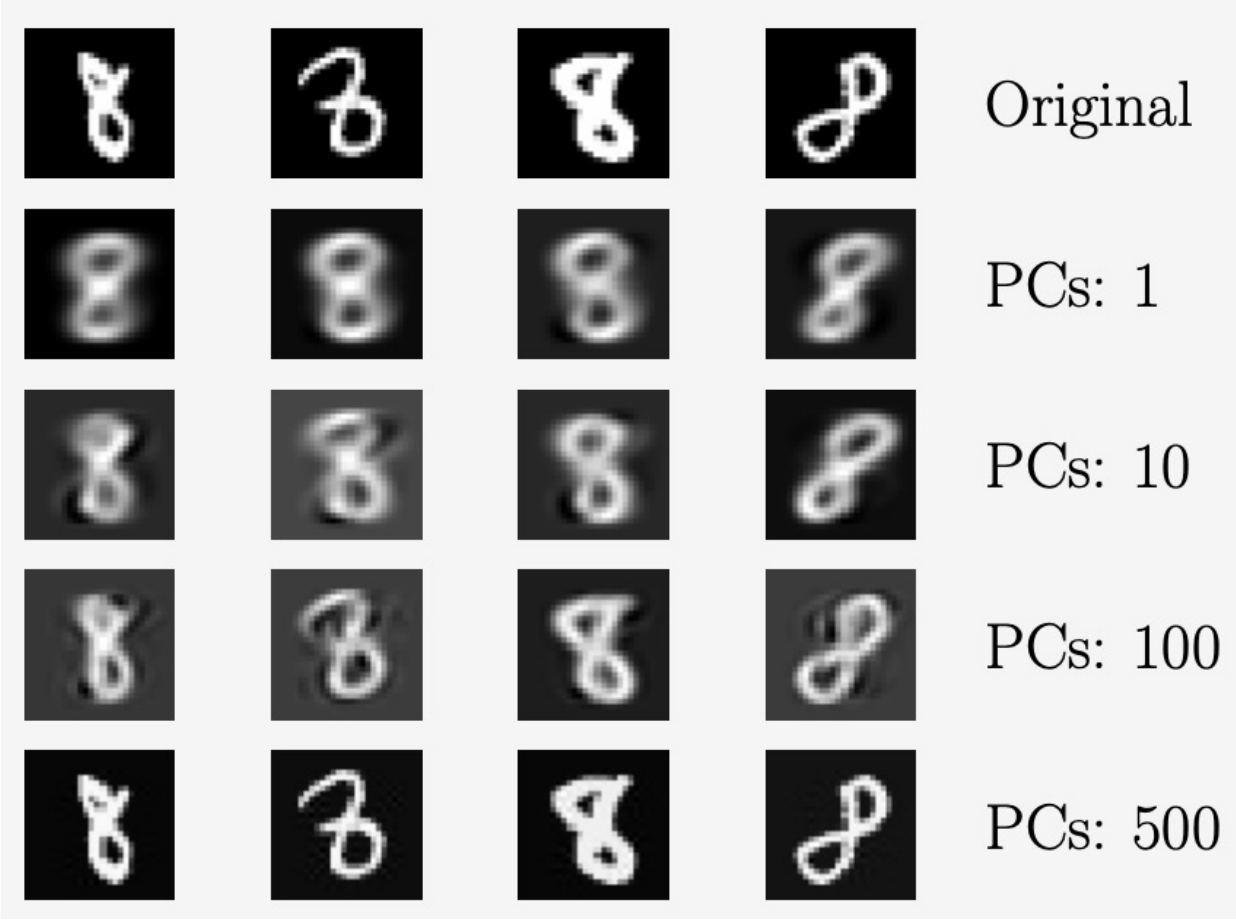
- Working directly with high-dimensional data, such as images, comes with some difficulties:
- It is hard to analyze,
- interpretation is difficult,
- visualization is nearly impossible,
- storage of the data vectors can be expensive.
- However, high-dimensional data often has properties that we can exploit.
- For example, high-dimensional data is often overcomplete, i.e., many dimensions are redundant and can be explained by a combination of other dimensions.
- Furthermore, dimensions in high-dimensional data are often correlated so that the data possesses an intrinsic lower-dimensional structure.

Introduction

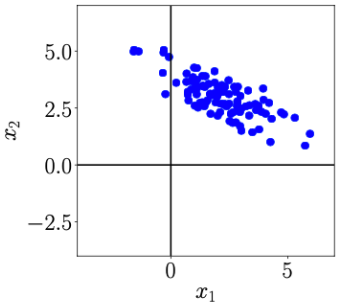
- Dimensionality reduction exploits structure and correlation and allows us to work with a more compact representation of the data, ideally without losing information.
- We can think of dimensionality reduction as a compression technique, similar to jpeg or mp3, which are compression algorithms for images and music.
- we will discuss *principal component analysis* (*PCA*), an algorithm for linear *dimensionality reduction*.
- PCA, proposed by Pearson (1901) and Hotelling (1933), has been around for more than 100 years and is still one of the most commonly used techniques for data compression and data visualization.
- It is also used for the identification of simple patterns, latent factors, and structures of high-dimensional data.

MNIST Digits: Reconstruction

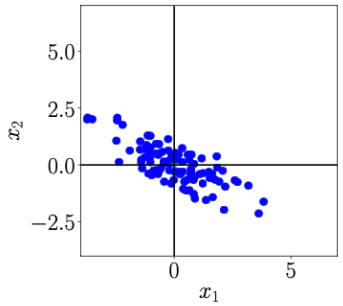
In the following, we will apply PCA to the MNIST digits dataset, which contains 60,000 examples of handwritten digits 0 through 9. Each digit is an image of size 28×28, i.e., it contains 784 pixels so that we can interpret every image in this dataset as a vector $\boldsymbol{x} \in \mathbb{R}^{784}$.



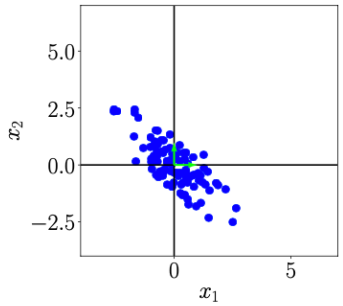
Key Steps of PCA in Practice



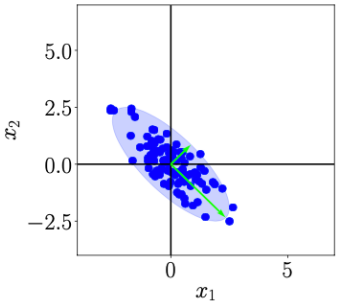
(a) Original dataset.



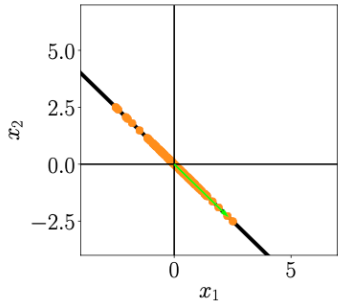
(b) Step 1: Centering by subtracting the mean from each data point.



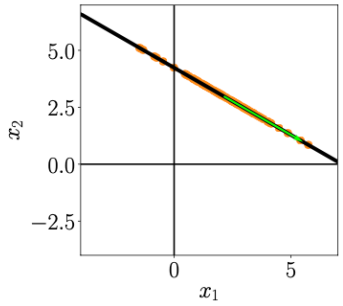
(c) Step 2: Dividing by the standard deviation to make the data unit free. Data has variance 1 along each axis.



(d) Step 3: Compute eigenvalues and eigenvectors (arrows) of the data covariance matrix (ellipse).



(e) Step 4: Project data onto the principal subspace.



(f) Undo the standardization and move projected data back into the original data space from (a).

- Since S is a symmetric matrix, it can be orthogonally diagonalized.
- This connection between statistics and linear algebra is the beginning of PCA.
- Let $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_m \geq 0$ be the eigenvalues of S with corresponding orthonormal eigenvectors $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_m$.
- These eigenvectors are called the **principal components** of the data set.
- Observation: on one hand, the trace of S is the sum of the diagonal entries of S, which is the sum of the variances of all m variables. Let's call this the total variance, T of the data.
- On the other hand, the trace of a matrix is equal the sum of its eigenvalues, so $T = \lambda_1 + \cdots + \lambda_m$.

The following interpretation is fundamental to PCA:

- The direction in $\mathbb{R}^m$ given by $\boldsymbol{u}_1$ (**the first principal direction or first principal component**) "explains" or "accounts for" an amount λ_1 of the total variance, T. The fraction of the total variance T accounts for is $\frac{\lambda_1}{T}$.
- And similarly, the **second principal direction** $\boldsymbol{u}_2$ accounts for the fraction $\frac{\lambda_2}{T}$ of the total variance, and so on.
- Thus, the vector $\boldsymbol{u}_1 \in \mathbb{R}^{\boldsymbol{m}}$ points in the most "significant" direction of the data set.
- Among directions that are orthogonal to $\boldsymbol{u}_1$, $\boldsymbol{u}_2$ points in the most "significant" direction of the data set.
- Among directions orthogonal to both $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$, the vector $\boldsymbol{u}_3$ points in the most significant direction, and so on.

Example:

Coming back to the facial recognition example. There are 5 faces involved, therefore the matrix A is of size 3×5

Face Width (x_1)	Face Height (x_2)	Nose Length (x_3)
14.0	20.0	5.5
13.5	19.5	5.2
13.8	20.2	5.6
14.2	20.5	5.7
13.9	19.8	5.3

So the matrix becomes $A = \begin{bmatrix} 14 & 20 & 5.5 \\ 13.5 & 19.5 & 5.2 \\ 13.8 & 20.2 & 5.6 \\ 14.2 & 20.5 & 5.7 \\ 13.9 & 19.8 & 5.3 \end{bmatrix}$

Example:

Then,

$$\boldsymbol{\mu} = [13.82 \quad 20.0 \quad 5.46]$$

Centralized Matrix

$$B = \begin{bmatrix} 0.18 & 0 & 0.04 \\ -0.32 & -0.5 & -0.26 \\ -0.32 & 0.2 & 0.14 \\ 0.38 & 0.5 & 0.24 \\ 0.08 & -0.2 & -0.16 \end{bmatrix}$$

Example:

To find the covariance matrix:

$$S = \frac{1}{4}B^T B = \begin{bmatrix} 0.097 & 0.0675 & 0.031 \\ 0.0675 & 0.145 & 0.0775 \\ 0.031 & 0.0775 & 0.043 \end{bmatrix}$$

Eigenvalues:

$$\lambda_1 = 0.236, \lambda_2 = 0.05, \lambda_3 = 0.001,$$

Eigenvectors:

$$\boldsymbol{u}_1 = \begin{bmatrix} 0.49 \\ 0.77 \\ 0.40 \end{bmatrix}, \boldsymbol{u}_2 = \begin{bmatrix} 0.87 \\ -0.39 \\ -0.30 \end{bmatrix}, \boldsymbol{u}_3 = \begin{bmatrix} 0.07 \\ -0.50 \\ 0.86 \end{bmatrix}$$

Example:

Eigenvalues:

$$\lambda_1 = 0.236, \lambda_2 = 0.05, \lambda_3 = 0.001,$$

Eigenvectors:

$$\boldsymbol{u}_1 = \begin{bmatrix} 0.49 \\ 0.77 \\ 0.40 \end{bmatrix}, \boldsymbol{u}_2 = \begin{bmatrix} 0.87 \\ -0.39 \\ -0.30 \end{bmatrix}, \boldsymbol{u}_3 = \begin{bmatrix} 0.07 \\ -0.50 \\ 0.86 \end{bmatrix}$$

So first principal component is $\boldsymbol{u}_1$, contributing 95% in the total variance, and second principal component is $\boldsymbol{u}_2$, contributing 4.8%. So,

$$P = [\boldsymbol{u}_1 \quad \boldsymbol{u}_2]$$

$$A_{reduced} = BP$$

$$= \begin{bmatrix} 0.104 & 0.145 \\ -0.646 & -0.0004 \\ 0.0532 & -0.4 \\ 0.668 & 0.0586 \\ -0.1788 & 0.1976 \end{bmatrix}$$

PCA Implementation

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Create a toy dataset (2D points)
X = np.array([
    [2.5, 2.4],
    [0.5, 0.7],
    [2.2, 2.9],
    [1.9, 2.2],
    [3.1, 3.0],
    [2.3, 2.7],
    [2.0, 1.6],
    [1.0, 1.1],
    [1.5, 1.6],
    [1.1, 0.9]
])

print("Original Data:\n", X)
```

PCA Implementation

```python
# 2. Standardize (zero mean)
X_meaned = X - np.mean(X , axis = 0)

# 3. Covariance matrix
cov_mat = np.cov(X_meaned , rowvar = False)
```

```python
print("C",cov_mat)
```

```
C [[0.61655556 0.61544444]
 [0.61544444 0.71655556]]
```

PCA Implementation

```
eigen_vals , eigen_vecs = np.linalg.eig(cov_mat)
print(eigen_vals,eigen_vecs)
sorted_idx = np.argsort(eigen_vals)[::-1]
eigen_vals = eigen_vals[sorted_idx]
eigen_vecs = eigen_vecs[:,sorted_idx]
print(sorted_idx,eigen_vals,eigen_vecs)
```

```
[0.0490834  1.28402771] [[-0.73517866 -0.6778734 ]
 [ 0.6778734  -0.73517866]]
[1 0] [1.28402771 0.0490834 ] [[-0.6778734  -0.73517866]
 [-0.73517866  0.6778734 ]]
```

PCA Implementation

```
pc1 = eigen_vecs[:,0]   # first principal compone
print(pc1)
X_pca = X_meaned @ pc1.reshape(-1,1)

print("\nEigenvalues:", eigen_vals)
print("Eigenvectors:\n", eigen_vecs)
print("\nProjected Data (1D):\n", X_pca)
```

PCA Implementation

```
[-0.6778734  -0.73517866]

Eigenvalues: [1.28402771 0.0490834 ]
Eigenvectors:
 [[-0.6778734  -0.73517866]
 [-0.73517866  0.6778734 ]]

Projected Data (1D):
 [[-0.82797019]
 [ 1.77758033]
 [-0.99219749]
 [-0.27421042]
 [-1.67580142]
 [-0.9129491 ]
 [ 0.09910944]
 [ 1.14457216]
 [ 0.43804614]
 [ 1.22382056]]
```

PCA Implementation

```
plt.figure(figsize=(7,5))
plt.scatter(X[:,0], X[:,1], label="Original Data")
plt.quiver(np.mean(X[:,0]), np.mean(X[:,1]), pc1[0], pc1[1],
           angles='xy', scale_units='xy', scale=1, color='r', label="PC1")
plt.xlabel("X1")
plt.ylabel("X2")
plt.title("PCA on Toy Data")
plt.legend()
plt.axis('equal')
plt.show()
```

PCA Implementation